

5.12.5

Dhanush Kumar A - AI25BTECH11010

October 3, 2025

Question

Solve for $x \in \mathbb{N}$ using matrices: $\det \begin{pmatrix} x+3 & -2 \\ -3x & 2x \end{pmatrix} = 8$

Solution

$$\det \begin{pmatrix} x+3 & -2 \\ -3x & 2x \end{pmatrix} = (x+3)(2x) - (-2)(-3x) \quad (1)$$

$$= 2x^2 + 6x - 6x \quad (2)$$

$$= 2x^2 \quad (3)$$

$$2x^2 = 8 \quad (4)$$

$$x^2 = 4 \quad (5)$$

$$x = \pm 2 \quad (6)$$

$$x \in \mathbb{N} \implies x = 2 \quad (7)$$

Python code - To solve

```
import numpy as np
from sympy import symbols, Eq, solve

# Define the variable
x = symbols('x')

# Define the matrix
A = np.array([[x + 3, -2],
              [-3*x, 2*x]])

# Compute the determinant using sympy
det = (x + 3)*(2*x) - (-2)*(-3*x)

# Equation: determinant = 8
equation = Eq(det, 8)
```

Python code - To solve

```
# Solve the equation
solutions = solve(equation, x)

# Filter for natural numbers
natural_solutions = [sol for sol in solutions if sol.is_real and
    sol > 0]

print("All solutions:", solutions)
print("Natural number solution(s):", natural_solutions)
```

Python code - Output

```
All solutions: [-2, 2]
```

```
Natural number solution(s): [2]
```

C code - Solving

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "/home/dhanush-kumar-a/ee1030-2025/ai25btech11010/matgeo
/5.12.5/codes/libs/matfun.h"

// Function to solve determinant equation  $\det\left(\begin{bmatrix} x+3 & -2 \\ -3x & 2 \end{bmatrix}\right) = 8$ 
double solve_det_equation() {
    double **A = createMat(2, 2);

    // We'll solve the equation  $2x^2 = 8$ , but let's use matrix
    // det
    // Represent matrix as  $\begin{bmatrix} x+3 & -2 \\ -3x & 2 \end{bmatrix}$ 
    // We'll solve symbolically by using quadratic formula
    double a = 2.0; // coefficient of  $x^2$ 
    double b = 0.0; // coefficient of  $x$ 
    double c = -8.0; // constant term
```

```
double x = roots[0][0]; // Return positive root (x > 0)

// Free allocated memory
free(A);
free(roots);

return x;
}

// Entry point for Python ctypes
double solve_det_ctypes() {
    return solve_det_equation();
}
```


Python code -Solving using c Funtion

```
import ctypes

lib = ctypes.CDLL('./libsolve_det.so') # Shared library compiled
    from above C code
lib.solve_det_ctypes.restype = ctypes.c_double

x = lib.solve_det_ctypes()
print("Solution x N:", x)
```

Python code-Output

```
Solution x N: 2.0
```